Canonical Universal Translation Pipeline (§ 1.1.4.1.3.3 . .)

This directory ships the **inherited, mandatory** translation standard for every project that inherits the HelixConstitution. Per **Constitution.md §11.4.133**, ALL translations of ANY content (UI strings, documents, articles, CVs, READMEs, marketing copy, in-app messages, release notes, legal text — any human-language string rendered in a non-source language) MUST be produced by **HelixTranslate** (`git@github.com:HelixDevelopment/HelixTranslate.git`) driven through the scripts in this directory. This is a hard, non-negotiable constraint — not a recommendation.

Purpose

A single, evidenced, fail-loud translation engine + pipeline so that every translated string in the fleet has a known provenance, a reproducible pipeline, and a captured accuracy proof. One engine, one pipeline, one evidence trail.

Files

Script	Purpose
`translate-pipeline.sh`	Universal Markdown / plain-text translator. Two modes: **plain** (whole file) and `--article` (YAML frontmatter kept byte-identical; only the body is translated, then reassembled).
`render-articles.sh`	Renders translated article Markdown sources into standalone HTML fragments (frontmatter `title` / `repo` / `tech` → header, body → `pandoc` html5).

Both scripts are **reusable verbatim** across projects and languages. Do not fork or hand-edit per project; configure them via environment variables (below).

Usage

```
# Translate a whole Markdown/plain file:
translate-pipeline.sh --in <src.md> --out <dst.md> --lang <ru|sr>

# Translate an article (preserve YAML frontmatter byte-identical, translate body):
translate-pipeline.sh --in <src.md> --out <dst.md> --lang <ru|sr> --article

# Render translated article sources to HTML fragments:
#   SOURCES: <site-root>/_article_src/<lang>/*.md
#   OUTPUT:  <site-root>/articles/<lang>/*.html
render-articles.sh <site-root> <lang> [<lang> ...]
```

Language → script mapping: `ru` → `cyrillic`, `sr` → `latin`.

Exit status: `0` only on a real, engine-produced translation. On failure the destination is **NOT written** and the script exits non-zero so callers detect it.

Provider configuration

Routing is **explicit and recorded** — never implicit/defaulted in the dark.

Variable	Default	Meaning
`HELIX_TRANSLATE_BIN`	`/Volumes/T7/Projects/helix_translate/build/unified-translator`	Path to the HelixTranslate `unified-translator` engine binary (built from the HelixTranslate repo by the consuming project).
`TRANSLATE_EVIDENCE_DIR`	`<project>/_tests/evidence/translate`	Where per-file run logs (evidence) are appended.

Canonical provider routing (in-script):

- **Primary:** provider `groq`, model `llama-3.3-70b-versatile`
- **Fallback:** provider `mistral`, model `mistral-large-latest`
- On rate-limit / error: retry up to 3× with linear backoff, then fail over to the fallback provider. The provider that actually produced the output is recorded in the per-file log.

Any deviation from this canonical routing MUST be stated explicitly in configuration and logged.

Hard rules (§ . .)

- **No hand-translation.** The engine is the single source of every translated string.
- **No other engines.** No Google Translate, DeepL, raw ChatGPT/Claude prompting, OS translation APIs, or library i18n auto-translators. Exactly one engine: HelixTranslate.
- **No silent fallback.** No quiet substitution of the source string, no empty/partial output passed off as a translation, no swallowed engine error, no "absence-of-error" PASS. On exhaustion of all providers the pipeline FAILS LOUD (non-zero exit, destination not written). Provider-to-provider failover **within** HelixTranslate (primary → fallback, with logged retries) is permitted and is NOT a silent fallback; falling back to any non-HelixTranslate source is forbidden.
- **Mandatory validation.** Every generated translation MUST be validated for accuracy with real (physical) captured evidence (per §11.4.5 / §11.4.69 / §11.4.107). The per-file run log (provider used, byte counts, retries, success/failure) is the evidence artifact. An unvalidated or unevidenced translation is treated as ABSENT.

Inherited standard

This is the **inherited** standard. Consuming projects MUST restate + cite §11.4.133 via §11.4.35 inheritance, supply their own source files / target languages, and build the HelixTranslate engine from `git@github.com:HelixDevelopment/`

`HelixTranslate.git`. Non-compliance is a release blocker. No escape hatch — no `--allow-hand-translation`, `--other-engine-OK`, `--skip-translation-validation`, or `--silent-fallback` flag exists.

See also: `docs/translations.md` and Constitution.md §11.4.133.